

Forward pass through a 2–3–1 network

Thirteen numbers, one example, one prediction. By the end of today every number inside the network on slide 21 has a name and a job, you have computed all of them with a pen, and three lines of NumPy agree with your paper to the third decimal.

Unit 2 · From scratch

80 minutes

Lab 9 · forward pass by hand

Calculator needed

The map

By hand

In NumPy

Naming

Lab 9

Before Friday — the hardest session of the term

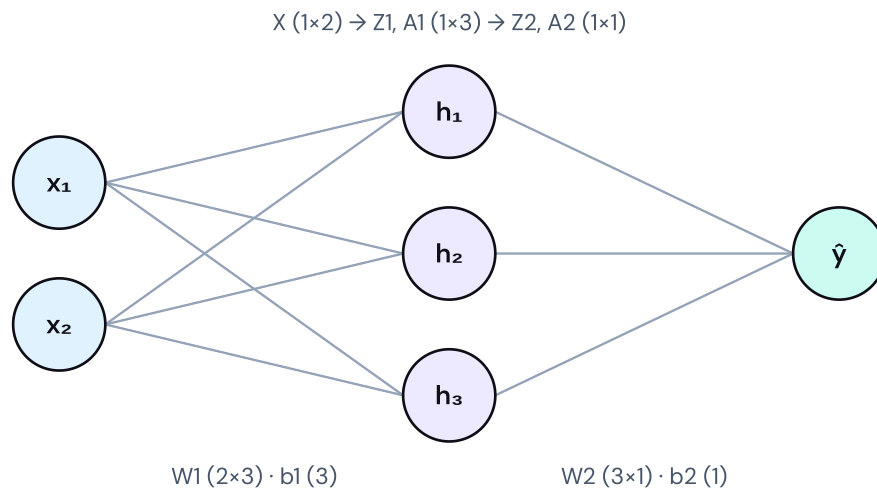
PART 1 · 10 MINUTES

The map of the network

Before any arithmetic: every array, its shape, and what one entry means.

1

DECK SLIDE 21, WITH THE SHAPES WRITTEN ON IT



Array	Shape	One entry means
<code>X</code>	(1, 2)	one example, two features
<code>W1</code>	(2, 3)	<code>W1[i, j]</code> : how strongly input i feeds hidden neuron j
<code>b1</code>	(3,)	one bias per hidden neuron
<code>z1</code> , <code>A1</code>	(1, 3)	each hidden neuron's score, then its activation
<code>W2</code>	(3, 1)	<code>W2[j, 0]</code> : how strongly hidden neuron j feeds the output
<code>b2</code>	(1,)	the output's bias
<code>z2</code> , <code>A2</code>	(1, 1)	the output score, then the prediction

ROWS ARE EXAMPLES, COLUMNS ARE NEURONS

Hold on to Lab 5's rule: in `X @ W1` , **column j of W1** collects everything flowing into hidden neuron j. Getting rows and columns crossed is the single most common error today, and it produces numbers that look perfectly plausible.

PART 2 · 25 MINUTES · PAPER FIRST

One example, every number by hand

The first layer is Lab 5's layer, unchanged. What is new is the squash after it and the second layer.

2

the network for today

TEXT

```
X = [[2.0, -1.0]]
W1 = [[ 1.0, 0.5, -2.0],
      [-1.0, 2.0,  0.0]]
W2 = [[1.0], [-2.0], [0.5]]
b1 = [0.5, -0.5, 1.0]
b2 = [0.2]
```

1 Hidden scores. Neuron 1: $2.0 \cdot 1.0 + (-1.0) \cdot (-1.0) + 0.5 = \mathbf{3.5}$. Neuron 2: $2.0 \cdot 0.5 + (-1.0) \cdot 2.0 - 0.5 = \mathbf{-1.5}$. Neuron 3: $2.0 \cdot (-2.0) + (-1.0) \cdot 0.0 + 1.0 = \mathbf{-3.0}$. You did exactly this in Lab 5.

- 2 **Hidden activations.** $\sigma(3.5) = \mathbf{0.971}$, $\sigma(-1.5) = \mathbf{0.182}$, $\sigma(-3.0) = \mathbf{0.047}$. One neuron strongly on, two mostly off.
- 3 **Output score.** $1.0 \cdot 0.971 + (-2.0) \cdot 0.182 + 0.5 \cdot 0.047 + 0.2 = 0.971 - 0.365 + 0.024 + 0.2 = \mathbf{0.830}$.
- 4 **Prediction.** $\sigma(0.830) = \mathbf{0.696}$. The network gives a 70% probability that this example is a yes.



The sign in the exponent

$\sigma(-1.5)$ must come out *below* 0.5. If yours is 0.818, you computed $1/(1 + e^{-1.5})$ — the sign of the score was lost. Check the easy fact first: negative score, probability under a half.

PART 3 · 15 MINUTES

The same pass in three lines

And then a whole batch, without changing a character.

3

Lab 9, task 2

PYTHON

```
def forward(X, W1, b1, W2, b2):
    Z1 = X @ W1 + b1          # (n, 3)
    A1 = sigmoid(Z1)          # (n, 3)
    Z2 = A1 @ W2 + b2          # (n, 1)
    A2 = sigmoid(Z2)           # (n, 1)
    return Z1, A1, Z2, A2
```

output

OUTPUT

```
Z1 [[ 3.5 -1.5 -3. ]]
A1 [[0.971 0.182 0.047]]
Z2 [[0.83]]
A2 [[0.696]]
```

Paper and code agree. Now the pay-off of two weeks of shapes: give `forward` the four XOR points as a (4, 2) array with Friday's hand-set weights, and it returns four predictions — `[0.001, 0.83, 0.83, 0.17]` — with no loop and no edit. Each row travels through the network independently and in parallel.

ASSERT, DO NOT EYEBALL

From today every hand computation is followed by an assertion: `assert np.allclose(A2, paper_A2, atol=2e-3)`. If it fails, **do not change the paper to match the code**. The disagreement is the most useful thing the exercise can give you.

PART 4 · 10 MINUTES

What every number is

Leave able to point at any one of the thirteen and say what it does.

4

Point and explain

Each answer is one sentence.

$W1[0, 2] = -2.0$. It multiplies input 1 on its way into hidden neuron 3; because input 1 is 2.0 here, it contributes -4.0 to that neuron's score and is the main reason neuron 3 is off.

$W2[1, 0] = -2.0$. It is how hidden neuron 2 votes on the output. A negative weight means: the more neuron 2 fires, the less likely a yes.

$b2 = 0.2$. The output's starting lean before any hidden neuron votes: with all three hidden neurons silent, the network would say $\sigma(0.2) = 0.55$.

If $W2[0, 0]$ doubled to 2.0, would A2 go up or down? Up: neuron 1 is at 0.971, so its vote grows by about 0.97, the output score rises to about 1.80, and $\sigma(1.80) \approx 0.86$.

LAB 9 · 18 MINUTES IN CLASS

Forward pass by hand

5

Lab 9 — `CSC4315_Lab09_Forward_Pass_By_Hand`. Paper first, then three lines, then the asserts; the parameter count comes from the arrays, not typed in.

Before Friday — the hardest session of the term

1. **Finish Lab 9.**
2. **Find your** `numgrad` from Lab 6. Friday is the session it was kept for.
3. **Revise the chain rule** from session 9: if u depends on v and v on w , then $du/dw = du/dv \cdot dv/dw$.



Session 14 — Friday 23 October, 11:30

Backpropagation, derived and coded. The same network, the same numbers, run backwards: how much would the loss change if we nudged each of the thirteen?